

CSC 3320: System-Level Programming



Chapter 13: Strings (contd.)

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Review: The *strlen* (String Length) Function

```
size_t strlen(const char *s);
```

- Returns the length of a string *s* (excluding the first null character).
- Example

```
int len = strlen("abc");  
/* return 3 */
```

Review: The *strcpy* (String Copy) Function

```
char *strcpy(char *dest,  
              const char *src);
```

- Copies the string `src` into the string `dest`.
- Example

```
str2 = "abcd";
```

```
strcpy(str2, "abcd");
```

```
strcpy(str1, str2);
```

```
strcpy(str1, strcpy(str2, "abcd"));
```

- It is safer to call `strncpy`, but it is slower.

```
char *strncpy(char * dest, const char * src, size_t s);
```

Review: The *strcat* (String Concatenate) Function

```
char *strcat(char *dest,  
              const char *src);
```

- Appends the contents of the string `src` to the end of the string `dest`, and return `dest`
- Example

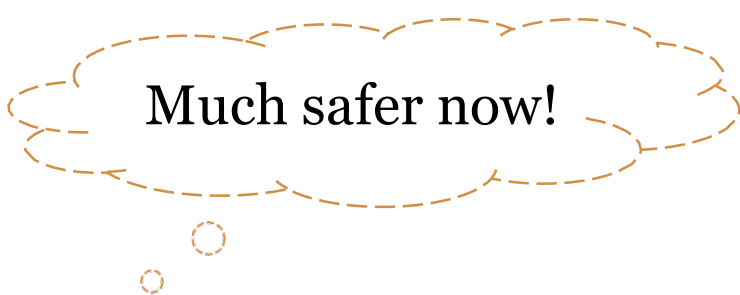
```
strcpy(str1, "abc");  
strcat(str1, "def");  
/* str1 now contains "abcdef" */
```

Review: The *strcat*(String Concatenate) Function

- Note: Do not forget how much space is left.
- Example

```
char str1[6] = "abc";  
strcat(str1, "def");    /*wrong*/
```

```
char str1[BUFFER_SIZE];  
char *str2="def";  
strncpy(s1, s2, BUFFER_SIZE - 1);  
strncat(s1, s2, BUFFER_SIZE-strlen(s1)-1);
```



Much safer now!

The *strcmp*(String Compare) Function

```
int strcmp(const char *dest,  
           const char *src);
```

- Compare `dest` and `src`, returning a value less than, equal to, or greater than 0 depending on whether `dest` is less than, equal to, or greater than `src`.

- Example

```
strcpy(str1, "abc");  
strcat(str2, "def");  
if(strcmp(str1, str2) < 0)  
    /* is str1 <= str2 ? True*/
```



Demonstration

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char src[] = "Hello";
    char dest1[10], dest2[10], dest3[10];

    strcpy(dest1, src); // Copies "Hello\0"
    strncpy(dest2, src, 3); // Copies "Hel", no null terminated
    strncpy(dest3, src, 6); // Copies "Hello" and null terminated, cause n > source length

    printf("strcpy: %s\n", dest1);
    printf("strncpy : %s\n", dest2); // Not null terminated, see the output
    printf("strncpy : %s\n", dest3); // Null terminated, see the output
    strcpy(dest2, "CSC3320");
    printf("strncpy : %s\n", dest2); //
    strncpy(dest2, src, 3); // Copies "Hel" and the rest of the destination would be the same
    printf("strncpy : %s\n", dest2);
    strcpy(dest1, "Hel"); // Copies "Hel\0", always null terminated
    printf("strcpy: %s\n", dest1);

    printf("%d\n", strcmp("Texas", "Dallas")); // Produces 1 if the first argument is larger
    printf("%d", strcmp("Dallas", "Texas")); // Produces -1 if the first argument is smaller

    return 0;
}
```

Output:

```
strcpy: Hello
strncpy : Hel+(some unwanted chars)
strncpy : Hello
strncpy : CSC3320
strncpy : Hel3320
strcpy: Hel
1
-1
```

Reading and Writing Strings

```
char str[] = "Are we having fun yet?";  
printf("%s\n", str);
```

The output will be

Are we having fun yet?

To print just part of a string, we can use the conversion specification `%.ps`,

Where ***p*** is the number of characters to be displayed. The statement.

```
printf("%.6s\n", str);
```

 will print Are we

More on Writing Strings

- The `%ms` conversion will display a string in a field of size m . String will not be truncated if $length > m$.
- Put a minus (–) sign before to specify left-justification.
- a conversion specification of the form `%m.ps` causes the first p characters of a string to be displayed in a field of size m .
- `puts(str)` is also used to write strings. The new-line is automatically added.

Reading Strings

- The %s conversion specification allows `scanf` to read a string into a character array:
`scanf("%s", str);`
- no need to put the & operator in front of `str` in the call of `scanf`: like any array name, `str` is treated as a pointer.
- Skips white spaces, reads, and then stops at white spaces.
- Alternative: use `scanf(" %99[^\n]", str);` OR `gets(str);`

What if the user enters: "Hello! World! I am a programmer!" to the function call

```
scanf("%s", str);
```

What value `str` will be assigned?

Output: "Hello!"

Some Useful Functions for Strings Handling

```
#include<ctype.h>
```

```
char ch = 'A';  
char ch2 = ' ';
```

```
isalnum(ch)    // returns true  
toupper(ch)    // keep it 'A'  
tolower(ch)    // make it a
```

Can you now convert any string to

- all lower-case letters
- and remove all non-alphanumeric characters?

Exercise 1

- The following function supposedly creates an identical copy of a string. What's wrong with the function?

```
char *duplicate(const char *p) {  
    char *q;  
  
    strcpy(q,p) ;  
    return q;  
}
```

Exercise 2

- What will be the value of the string *s1* at the end?

```
strcpy(s1, "computer");  
Strcpy(s2, "science");
```

```
if(strcmp(s1, s2) < 0)  
    strcat(s1, s2);  
else  
    strcat(s2, s1);
```

```
s1[strlen(s1) - 6] = '\\0';
```